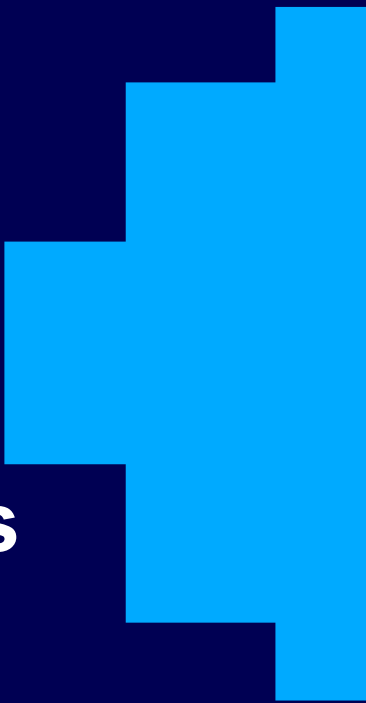
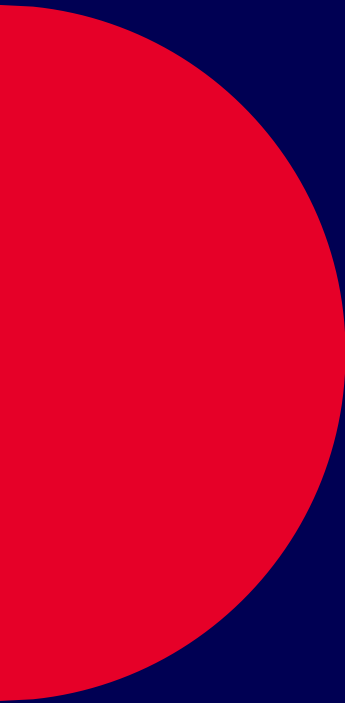




Games & AI Techniques

Week 3:

Finite State Machine in Games

Dr. Ginel Dorleon

Week 1-2 Recap

About this course

This course is about:

- AI Techniques applied to games
- We will cover more AI techniques than talking about games (80/20)
- We will explain you different AI Techniques and how they can be applied in some old school games
- Mastering the AI Techniques is the most important aspect of the course.

What this course is not about ?

- High level of Gaming Dev **X**
- This course is not about gaming **X**

What do I have to do in this course ?

Assignment:

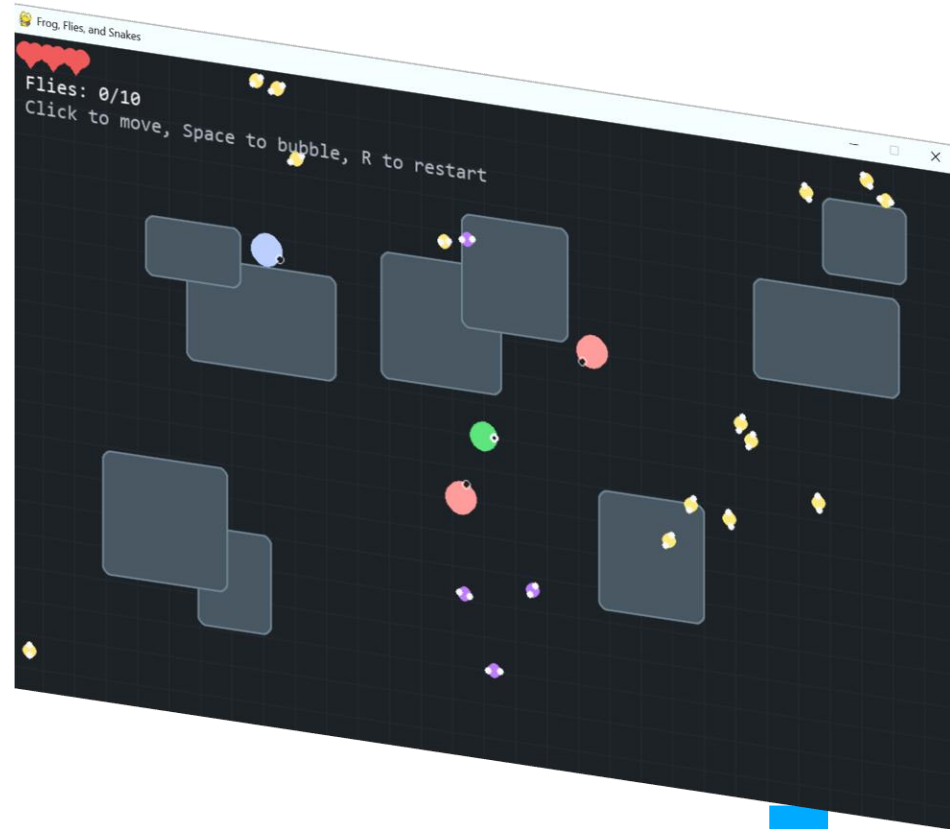
- You must complete 3 Assignments: 2 Individually, 1 in a group of 3-4
- Each assignment has a Technical Interview (we want to make sure you understand the techniques)
- 3 assignments => 5+ AI Techniques
- **A1: Steering Behaviors & FSM**, **A2: Path Planning & MCTS**, **A3: Reinforcement Learning**

Participation:

- Attend online lectures and the face-to-face tutorial. Ask question !!!
- Join discussion and build on others. Read extra source each week.
- **Self learning:** read beyond class, review after class. Apply learned concepts to assignments.

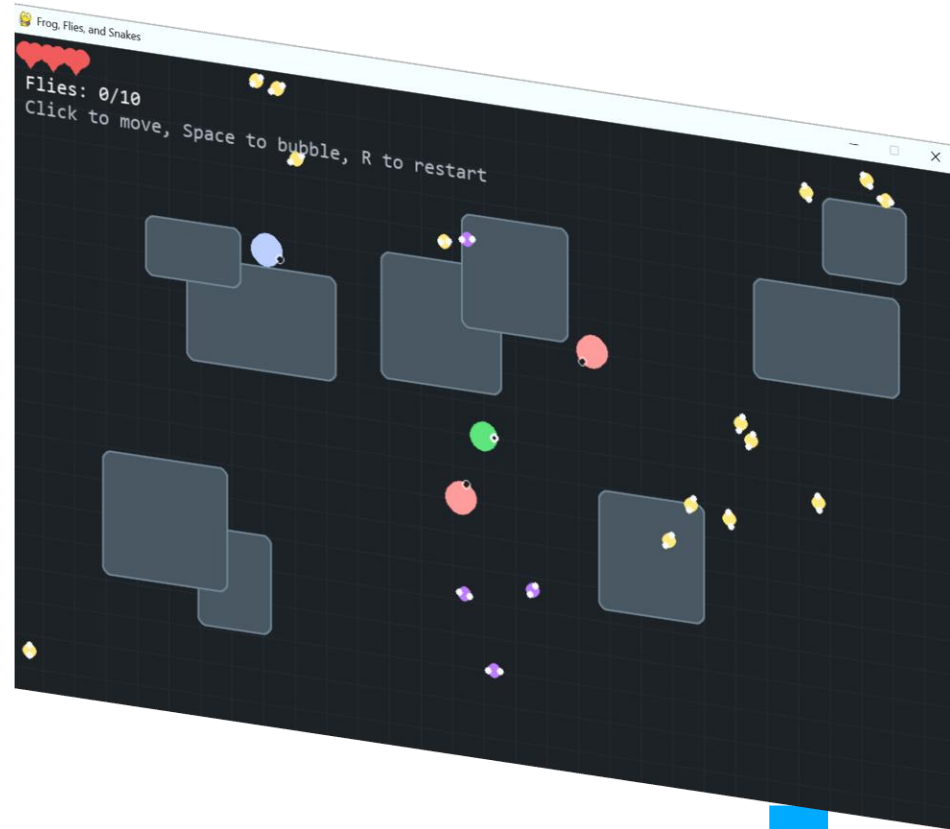
Assignment 1 is available on Canvas

- Predator/prey game with frog, flies, snakes.
- Focus is on old-school games + AI: **Steering behaviours + finite state machines.**
- A little bit game dev, since this is *games* and AI after all.
- Graphics, animations, and **some skeleton code is provided**. You need to implement the missing bits.
- Due at the end of week 4.
- Expectation: Do one of the 3 parts each week (starting today).



Assignment 1 is available on Canvas

- Assignment 1 : Deadline is November 23rd
- Technical Interview is on week 4 – During Tutorial



Week 2 -

- Steering Behaviors?
- The Physics Behind Steering
- Types of Steering Behaviors



What is Steering?

Steering behavior is a rule that tells a game character how to move in a way that looks natural and intelligent, based on what it sees or wants to do.

Why We Use Steering in Games

Steering is used whenever we want believable motion

Situation	Example in games	Steering type used
Move toward a goal	Frog moves to a clicked point	Seek
Move away from danger	Fly runs from snake	Flee
Stop smoothly at target	Frog slows near fly	Arrival
Chase a moving target	Snake hunts frog	Pursuit
Wander naturally	Flies buzz randomly	Wander
Avoid bumping into others	Snakes don't collide	Separation / Avoidance
Group movement	Birds flock together	Flocking



The Physics Behind Steering



The Tiny Physics Behind Steering

Understand how movement works in a game world.

Everything that moves has 3 things/3 basic quantities:

- Every moving game object: a frog, a car, a spaceship, can be described by just 3 basic quantities:

	Symbol	Question it answers	Example
Position	p	"Where am I?"	(100, 200) on the screen
Velocity	v	"How am I moving?"	5 pixels per frame to the right
Acceleration	a	"How is my movement changing?"	speeding up, slowing down..

The Tiny Physics Behind Steering

How Movement Happens?

Imagine driving a car:

- **Position:** where your car is right now on the road.
- **Velocity:** how fast and in what direction we're driving/the speed of something given a direction!
- **Acceleration:** how your speed changes: pressing gas, braking, or turning the wheel.

Now in games, we update these things many times every second => frames per second or FPS.

- Acceleration changes velocity.
- Velocity changes position.

- **FPS:** how often the game updates each second.
- **Δ time** (delta time): the amount of time that passes between two frames(two updates).
- **Δt** is the inverse of **FPS** => **$\Delta t = 1 / \text{FPS}$**

The Tiny Physics Behind Steering



The Formulas:

For each frame, we do:

new_velocity = old_velocity + acceleration * dt

new_position = old_position + velocity * dt

Remember: dt means “delta time”, how many seconds have passed since the last frame (usually around 1/60 of a second).



The Tiny Physics Behind Steering

The Role of Force (What Causes Acceleration)

Now we ask: what makes acceleration happen?

In real life, forces change our velocity:

- Gas pedal pushes the car forward.
- Brakes apply force backward.
- Turning the wheel changes the direction of the force.



In our game, steering behaviors are those forces!

- Each steering behavior computes a **steering force**: a small vector that says “turn this way” or “slow down a bit”.
- Then we convert that force into acceleration:
$$\text{acceleration} = \text{steering_force} / \text{mass}$$
if we treat mass = 1 (to keep things simple), then:
$$\text{acceleration} = \text{steering_force}$$





Week3 – Finite State Machine in Games



Finite State Machine

- A finite State Machine (FSM) is a simple computation model that moves among a limited set of states in response to inputs.

Simply:

- **A simple computational model** that has **a fixed, known set of states, lives in exactly one state at a time, reads input symbols one by one, and moves between states by following rules.**
- It starts in a designated start state. Some states are marked accepting. When the input ends, the machine says yes if it sits in an accepting state, otherwise no.

Finite State Machine in Games



Why do we need something like FSM?

- Game characters must decide what to do based on the world around them. We want behavior that is fast, believable, and easy to debug.
- Different AI tools exist, and FSM is a simple, reliable one for many cases.



Finite State Machine in Games



An FSM for a videogame character consists of:

- **States**, which define the character's various behaviours. A character is always in one state (never zero, never more than one).
- An **initial state**.
- **Events**, which cause transitions between states.
- A transition function that says where to go next when an event happens
- A character is always in exactly one state at a time

Finite State Machine in Games

A Finite State Machine is a small set of named states plus rules that tell you when to switch from one state to another.

Example:

An enemy with 2 states:

PATROL - wander around randomly.

CHASE - move toward the player if they are close enough.

The rules are:

- If the enemy is in PATROL and the player comes close, switch to CHASE.
- If the enemy is in CHASE and the player gets far away, switch back to PATROL.

Finite State Machine in Games



A Finite State Machine has:

- A set of states, like Wander, Chase(seek), Flee
- One initial state
- Inputs or events that can occur
- A transition function that says where to go next when an event happens
- A character is always in exactly one state at a time



Finite State Machine in Games

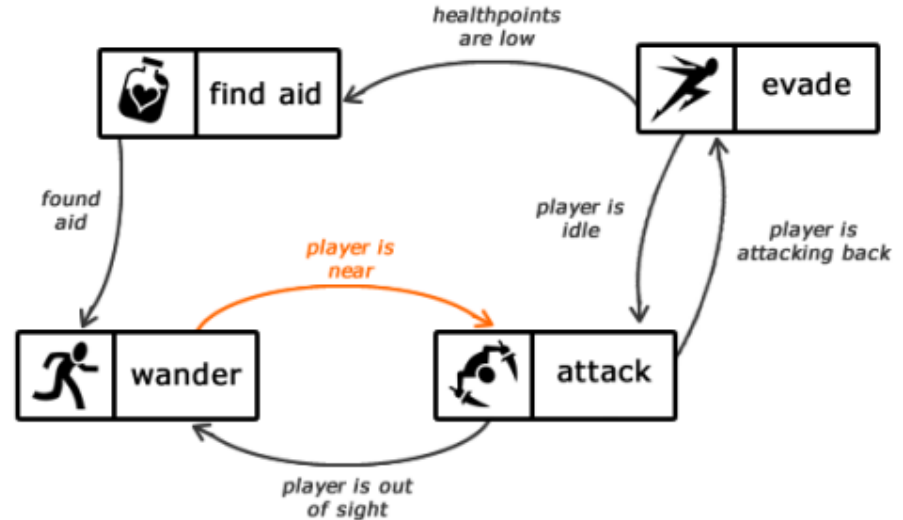
Informal FSM Diagram

States: initial state

- Wander - Attack - Evade - First Aid

Transition and Triggers:

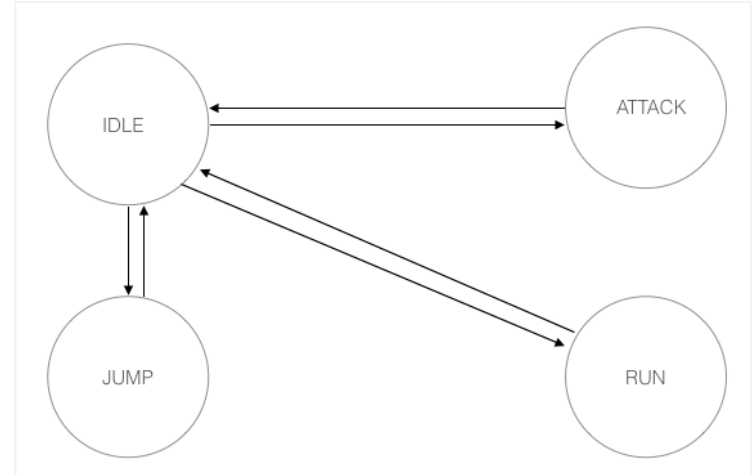
- wander** → **attack** when the player is near (orange arrow highlights this key trigger).
- attack** → **wander** when the player is out of sight.
- attack** → **evade** when the player is attacking back.
- evade** → **attack** when the player is idle.
- evade** → **find aid** when healthpoints are low.
- find aid** → **wander** after it found aid.



Finite State Machine in Games

Diagrams that tell a story:

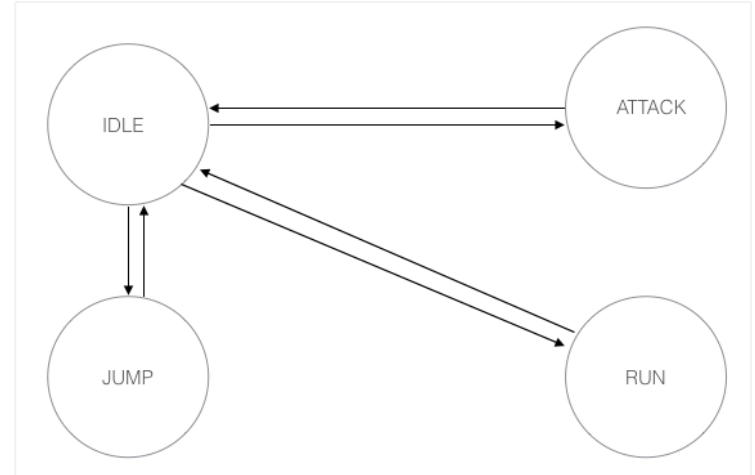
- Use circles/square... for states and arrows for transitions
For example: Wander might use your existing steering behavior for wandering, and Chase might use seek or pursue.
- Transitions switch which behavior is active.
- Keep behavior code separate from transition logic so things stay modular.



Finite State Machine in Games

Diagrams that tell a story:

- IDLE → ATTACK: attack
- ATTACK → IDLE: attack_done
- IDLE → RUN: move_on
- RUN → IDLE: move_off
- IDLE → JUMP: jump
- JUMP → IDLE: landed



Finite State Machine



From Steering Behavior to States

Each state plugs into one steering behavior:

- Wander state calls wander steering
- Chase state calls seek or pursue
- Flee state calls flee
- This is simple behavior switching rather than blending. You can still blend if you want inside a state.

Two common FSM Styles:

- Moore style: actions live in states, so you run code each update while you are in that state
- Mealy style: actions occur on transitions

Game devs usually find Moore more intuitive.





Finite State Machine in Games

What must you deliver for FSMs in assignment 1 ?

Finite State Machine in Games

Fly FSM : States and Triggers (Assignment 1)

- States: Flock and Fleeing.
- Fleeing triggers when frog or bubbles enter a danger radius.
- Add a timer so flies do not flicker between states.
- What to implement: distance checks to frog and bubbles, Flock → Fleeing → Flock flow, optional easing timer

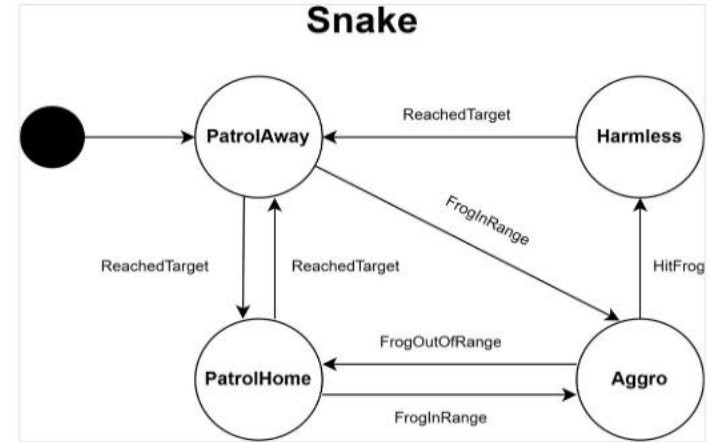
```
if state is FLOCK and danger_near():  
    state = FLEEING  
    scared_until = now + 1.0  
elif state is FLEEING:  
    if now >= scared_until and safe_again():  
        state = FLOCK
```

Finite State Machine in Games

States: PatrolAway, PatrolHome, Aggro, Harmless

Initial state: PatrolAway

Events: FrogInRange, FrogOutOfRange, ReachedTarget, HitFrog



Current state	FrogInRange	FrogOutOfRange	ReachedTarget	HitFrog
PatrolAway	Aggro	PatrolAway	PatrolHome	PatrolAway
PatrolHome	Aggro	PatrolHome	PatrolAway	PatrolHome
Aggro	Aggro	PatrolHome	Aggro	Harmless
Harmless	Harmless	Harmless	PatrolAway	Harmless

Finite State Machine in Games

Snake Transitions logic you might decide to implement:

- Turn patrol into chase when inside AggroRange.
- Drop aggro when the frog exits DeAggroRange.
- When chase ends or pacified, return home using arrive or seek as instructed.

```
if state in {PATROL_AWAY, PATROL_HOME} and dist_to_frog < AGGRO_RANGE:  
    state = AGGRO  
elif state is AGGRO and dist_to_frog > DEAGGRO_RANGE:  
    state = PATROL_HOME
```

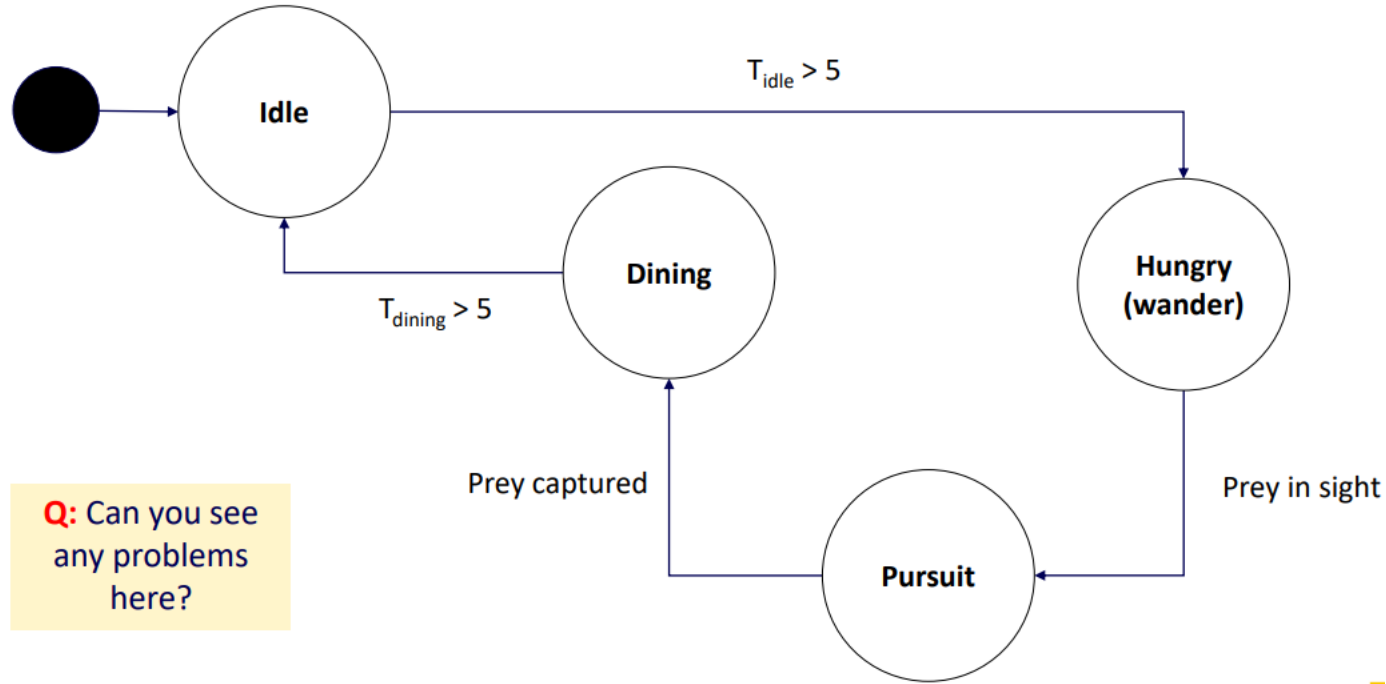
Finite State Machine in Games

Plugging Steering into States

- PatrolAway and PatrolHome call arrive toward their targets.
- Aggro calls seek at first, then pursue after you add the extension.
- Harmless heads home with arrive.
- Flock: separation plus cohesion plus alignment
- Fleeing: add flee, increase its weight near the threat

Finite State Machine in Games

Avoiding Problem in Designing your FSM Diagram



Finite State Machine in Games

Avoiding Problem in Designing your FSM Diagram

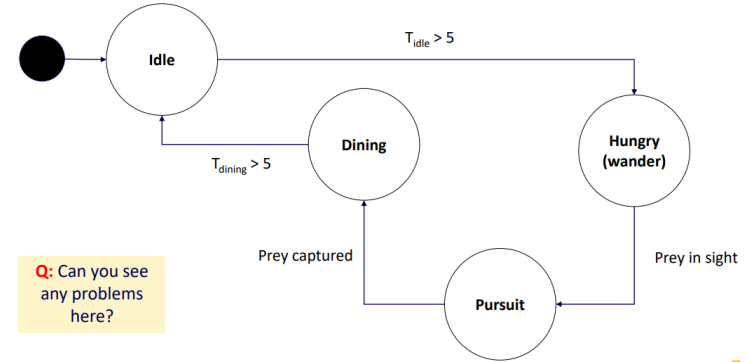
This diagram shows four states for some creature or agent:

Idle → Hungry → Pursuit → Dining → Idle

The arrows show when it switches between them:

- After being Idle for more than 5 units of time, it becomes Hungry.
- When it sees prey, it starts Pursuit.
- When it captures prey, it goes to Dining.
- After dining for more than 5 units of time, it goes back to Idle.

There is an issue here, what is it ?



Finite State Machine in Games

Avoiding Problem in Designing your FSM Diagram

This diagram shows four states for some creature or agent:

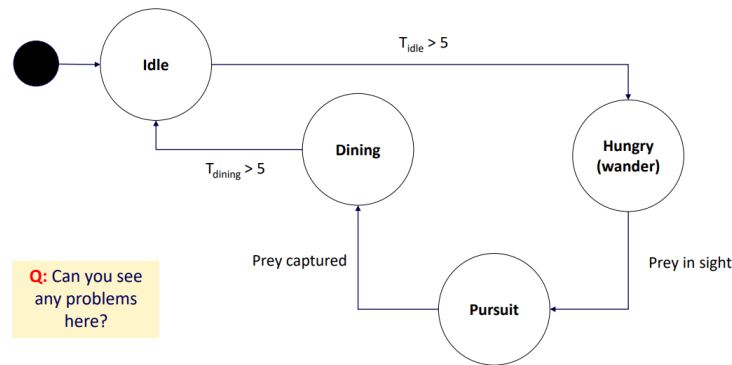
Idle → Hungry → Pursuit → Dining → Idle

The arrows show when it switches between them:

- After being Idle for more than 5 units of time, it becomes Hungry.
- When it sees prey, it starts Pursuit.
- When it captures prey, it goes to Dining.
- After dining for more than 5 units of time, it goes back to Idle.

There is an issue here, what is it ?

- Look at the Pursuit state. There's only one arrow out of it, labeled Prey captured, which leads to Dining.
- So what happens if the prey escapes or is lost from sight?



Q: Can you see any problems here?

Finite State Machine in Games

LEARNING ACTIVITY I

Learning Activity

Problem Scenario

You are controlling a simple snake AI with a tiny FSM. The FSM has three states: **Patrol**, **Aggro**, and **Harmless**. The machine starts in Patrol. From Patrol, if the frog enters the detection radius, the event FrogInRange occurs, and the snake switches to Aggro. From Aggro, if the frog moves outside the de-aggro radius, the event FrogOutOfRange occurs, and the snake returns to Patrol. From Aggro, if a bubble hits the snake, the event HitByBubble occurs, and the snake becomes Harmless. From Harmless, when the snake reaches its Home position, the event ReachedHome occurs, and it goes back to Patrol. If no event that matches an outgoing arrow is true on a given frame, the FSM stays in the current state.

States: Patrol, Aggro, Harmless
Start: Patrol

Transitions:

Patrol → Aggro on FrogInRange

Aggro → Patrol on FrogOutOfRange

Aggro → Harmless on HitByBubble

Harmless → Patrol on ReachedHome

If nothing matches, stay in the same state.

Learning Activity

Problem Scenario

You are controlling a simple snake AI with a tiny FSM. The FSM has three states: **Patrol**, **Aggro**, and **Harmless**. The machine starts in Patrol. From Patrol, if the frog enters the detection radius, the event `FrogInRange` occurs, and the snake switches to Aggro. From Aggro, if the frog moves outside the de-aggro radius, the event `FrogOutOfRange` occurs, and the snake returns to Patrol. From Aggro, if a bubble hits the snake, the event `HitByBubble` occurs, and the snake becomes Harmless. From Harmless, when the snake reaches its Home position, the event `ReachedHome` occurs, and it goes back to Patrol. If no event that matches an outgoing arrow is true on a given frame, the FSM stays in the current state.

Q&A:

Q1. What is the initial state?

- A. Patrol
- B. B. Aggro
- C. C. Harmless
- D. D. None

A: Patrol

Learning Activity

Problem Scenario

You are controlling a simple snake AI with a tiny FSM. The FSM has three states: **Patrol**, **Aggro**, and **Harmless**. The machine starts in Patrol. From Patrol, if the frog enters the detection radius, the event FrogInRange occurs, and the snake switches to Aggro. From Aggro, if the frog moves outside the de-aggro radius, the event FrogOutOfRange occurs, and the snake returns to Patrol. From Aggro, if a bubble hits the snake, the event HitByBubble occurs, and the snake becomes Harmless. From Harmless, when the snake reaches its Home position, the event ReachedHome occurs, and it goes back to Patrol. If no event that matches an outgoing arrow is true on a given frame, the FSM stays in the current state.

Q&A:

Q2. From Patrol, FrogInRange happens. Next state?

- A. Patrol
- B. B. Aggro
- C. C. Harmless
- D. D. No change defined

Aggro

Learning Activity

Problem Scenario

You are controlling a simple snake AI with a tiny FSM. The FSM has three states: **Patrol**, **Aggro**, and **Harmless**. The machine starts in Patrol. From Patrol, if the frog enters the detection radius, the event `FrogInRange` occurs, and the snake switches to Aggro. From Aggro, if the frog moves outside the de-aggro radius, the event `FrogOutOfRange` occurs, and the snake returns to Patrol. From Aggro, if a bubble hits the snake, the event `HitByBubble` occurs, and the snake becomes Harmless. From Harmless, when the snake reaches its Home position, the event `ReachedHome` occurs, and it goes back to Patrol. If no event that matches an outgoing arrow is true on a given frame, the FSM stays in the current state.

Q&A:

Q3. From Aggro, `FrogOutOfRange` happens. Next state?

- A. Patrol
- B. B. Aggro
- C. C. Harmless
- D. D. No change defined

Patrol

Learning Activity

Problem Scenario

You are controlling a simple snake AI with a tiny FSM. The FSM has three states: **Patrol**, **Aggro**, and **Harmless**. The machine starts in Patrol. From Patrol, if the frog enters the detection radius, the event FrogInRange occurs, and the snake switches to Aggro. From Aggro, if the frog moves outside the de-aggro radius, the event FrogOutOfRange occurs, and the snake returns to Patrol. From Aggro, if a bubble hits the snake, the event HitByBubble occurs, and the snake becomes Harmless. From Harmless, when the snake reaches its Home position, the event ReachedHome occurs, and it goes back to Patrol. If no event that matches an outgoing arrow is true on a given frame, the FSM stays in the current state.

Q&A:

Q4. From Aggro, HitByBubble happens. Next state?

- A. Patrol
- B. Aggro
- C. Harmless
- D. No change defined

Harmless

Learning Activity

Problem Scenario

You are controlling a simple snake AI with a tiny FSM. The FSM has three states: **Patrol**, **Aggro**, and **Harmless**. The machine starts in Patrol. From Patrol, if the frog enters the detection radius, the event FrogInRange occurs, and the snake switches to Aggro. From Aggro, if the frog moves outside the de-aggro radius, the event FrogOutOfRange occurs, and the snake returns to Patrol. From Aggro, if a bubble hits the snake, the event HitByBubble occurs, and the snake becomes Harmless. From Harmless, when the snake reaches its Home position, the event ReachedHome occurs, and it goes back to Patrol. If no event that matches an outgoing arrow is true on a given frame, the FSM stays in the current state.

Q&A:

Q5. From Harmless, ReachedHome happens. Next state?

- A. Patrol
- B. Aggro
- C. Harmless
- D. No change defined

Patrol

Learning Activity

Problem Scenario

You are controlling a simple snake AI with a tiny FSM. The FSM has three states: **Patrol**, **Aggro**, and **Harmless**. The machine starts in Patrol. From Patrol, if the frog enters the detection radius, the event FrogInRange occurs, and the snake switches to Aggro. From Aggro, if the frog moves outside the de-aggro radius, the event FrogOutOfRange occurs, and the snake returns to Patrol. From Aggro, if a bubble hits the snake, the event HitByBubble occurs, and the snake becomes Harmless. From Harmless, when the snake reaches its Home position, the event ReachedHome occurs, and it goes back to Patrol. If no event that matches an outgoing arrow is true on a given frame, the FSM stays in the current state.

Q&A:

Q6. From Patrol, ReachedHome happens. Next state?

- A. No changes
- B. Aggro
- C. Harmless
- D. Patrol

Patrol

Learning Activity

Problem Scenario

You are controlling a simple snake AI with a tiny FSM. The FSM has three states: **Patrol**, **Aggro**, and **Harmless**. The machine starts in Patrol. From Patrol, if the frog enters the detection radius, the event FrogInRange occurs, and the snake switches to Aggro. From Aggro, if the frog moves outside the de-aggro radius, the event FrogOutOfRange occurs, and the snake returns to Patrol. From Aggro, if a bubble hits the snake, the event HitByBubble occurs, and the snake becomes Harmless. From Harmless, when the snake reaches its Home position, the event ReachedHome occurs, and it goes back to Patrol. If no event that matches an outgoing arrow is true on a given frame, the FSM stays in the current state.

Q&A:

Q7. Which item below is a state, not an event?

- A. FrogInRange
- B. HitByBubble
- C. ReachedHome
- D. Aggro

Aggro

Learning Activity

Problem Scenario

You are controlling a simple snake AI with a tiny FSM. The FSM has three states: **Patrol**, **Aggro**, and **Harmless**. The machine starts in Patrol. From Patrol, if the frog enters the detection radius, the event FrogInRange occurs, and the snake switches to Aggro. From Aggro, if the frog moves outside the de-aggro radius, the event FrogOutOfRange occurs, and the snake returns to Patrol. From Aggro, if a bubble hits the snake, the event HitByBubble occurs, and the snake becomes Harmless. From Harmless, when the snake reaches its Home position, the event ReachedHome occurs, and it goes back to Patrol. If no event that matches an outgoing arrow is true on a given frame, the FSM stays in the current state.

Q&A:

Q8. Which label correctly belongs on the arrow from Aggro to Harmless?

- A. FrogOutOfRange
- B. Patrol
- C. ReachedHome
- D. HitByBubble

HitByBubble

Learning Activity

Problem Scenario

You are controlling a simple snake AI with a tiny FSM. The FSM has three states: **Patrol**, **Aggro**, and **Harmless**. The machine starts in Patrol. From Patrol, if the frog enters the detection radius, the event **FrogInRange** occurs, and the snake switches to Aggro. From Aggro, if the frog moves outside the de-aggro radius, the event **FrogOutOfRange** occurs, and the snake returns to Patrol. From Aggro, if a bubble hits the snake, the event **HitByBubble** occurs, and the snake becomes Harmless. From Harmless, when the snake reaches its Home position, the event **ReachedHome** occurs, and it goes back to Patrol. If no event that matches an outgoing arrow is true on a given frame, the FSM stays in the current state.

Q&A:

Q9. If no transition condition is true this frame, what should the FSM do?

- A. Jump to the initial state
- B. Pick a random state
- C. Stay in the current state
- D. Stop the game loop

Stay in the current state



End of the Lecture – Thank you!

See you in class for tutorial.